

2 UEFI Secure Boot

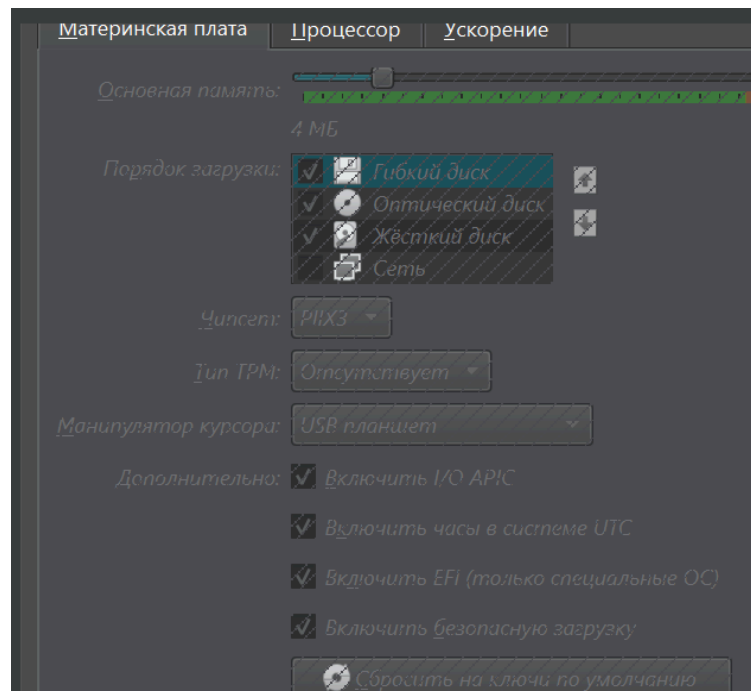
Performed by *Nikita Niakhai*

▼ Pipeline

Task 1 - Firmware Databases

▼ step 1

Turn on UEFI mode on my Virtual Box machine





Because I have installed my machine simply in Legacy Mode (BIOS), I am obliged to reinstall in UEFI mode



I used all hints to find proper tools for extracting and viewing the certificate

Install tools to analyse certificates

```
nikita@DEMA: ~  
File Actions Edit View Help  
(nikita@DEMA)-[~]  
$ sudo apt install efityls
```

```
sudo apt install efityls  
sudo apt install openssl
```

Dump firmware vars (db = allowed keys database)

```
sudo efivar -v db -o db.auth
```

Extract certs from the db file

```
cert-to-efi-sig-list db.auth db.esl  
sig-list-to-certs db.esl certs_dir/
```

```
(nikita@nikita)-[~]  
$ sudo efi-readvar -v db -o db.esl  
Variable db, length 6133  
  
(nikita@nikita)-[~]  
$ ls -lh db.esl  
-rw-rw-r-- 1 nikita nikita 6.0K Sep 15 09:51 db.esl
```

```
Variable db, length 6133  
  
(nikita@nikita)-[~]  
$ cert-to-efi-sig-list db.auth db.esl
```

```
(nikita@nikita)-[~]  
$ mkdir -p certs_dir  
  
(nikita@nikita)-[~]  
$ sig-list-to-certs db.esl certs_dir  
X509 Header sls=1600, header=0, sig=1556  
file certs_dir-0.der: Guid 77fa9abd-0359-4d32-bd60-28f4e78f784b  
Written 1556 bytes  
X509 Header sls=1492, header=0, sig=1448  
file certs_dir-1.der: Guid 77fa9abd-0359-4d32-bd60-28f4e78f784b  
Written 1448 bytes  
X509 Header sls=1543, header=0, sig=1499  
file certs_dir-2.der: Guid 77fa9abd-0359-4d32-bd60-28f4e78f784b  
Written 1499 bytes  
X509 Header sls=1498, header=0, sig=1454  
file certs_dir-3.der: Guid 77fa9abd-0359-4d32-bd60-28f4e78f784b  
Written 1454 bytes
```

```
(nikita@nikita)-[~]  
$ cd certs_dir  
  
(nikita@nikita)-[~/certs_dir]  
$ ls  
-0.der  certs_dir-0.der  certs_dir-1.der  certs_dir-2.der  certs_dir-3.der
```

Viewing the certificate with `openssl x509`

```
for f in certs_dir/*.der; do  
    echo "==== $f ====="  
    openssl x509 -in "$f" -inform DER -noout -subject -issuer -text | sed -
```

n '1,20p'
done

```
(nikita@nikita)-[~]
$ for f in certs_dir/*.der; do
  echo "==== $f ====="
  openssl x509 -in "$f" -inform DER -noout -subject -issuer -text | sed -n '1,
20p'
done
==== certs_dir/-0.der =====
Could not find certificate from certs_dir/-0.der
==== certs_dir/certs_dir-0.der =====
subject=C=US, ST=Washington, L=Redmond, O=Microsoft Corporation, CN=Microsoft
Corporation UEFI CA 2011
issuer=C=US, ST=Washington, L=Redmond, O=Microsoft Corporation, CN=Microsoft C
orporation Third Party Marketplace Root
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      61:08:d3:c4:00:00:00:00:00:04
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: C=US, ST=Washington, L=Redmond, O=Microsoft Corporation, CN=Mi
crosoft Corporation Third Party Marketplace Root
    Validity
      Not Before: Jun 27 21:22:45 2011 GMT
      Not After : Jun 27 21:32:45 2026 GMT
    Subject: C=US, ST=Washington, L=Redmond, O=Microsoft Corporation, CN=M
icrosoft Corporation UEFI CA 2011
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      Public-Key: (2048 bit)
      Modulus:
        00:a5:08:6c:4c:c7:45:09:6a:4b:0c:a4:c0:87:7f:
        06:75:0c:43:01:54:64:e0:16:7f:07:ed:92:7d:0b:
        b2:73:bf:0c:0a:c6:4a:45:61:a0:c5:16:2d:96:d3:
      Exponent: 65537 (0x10001)
    ===== certs_dir/certs_dir-1.der =====
```

==== certs_dir/certs_dir-0.der =====
subject=C=US, ST=Washington, L=Redmond, O=Microsoft Corporation,
CN=Microsoft Corporation UEFI CA 2011
issuer=C=US, ST=Washington, L=Redmond, O=Microsoft Corporation,
CN=Microsoft Corporation Third Party Marketplace Root
Certificate:
Data:
Version: 3 (0x2)
Serial Number:
61:08:d3:c4:00:00:00:00:00:04
Signature Algorithm: sha256WithRSAEncryption
Issuer: C=US, ST=Washington, L=Redmond, O=Microsoft Corporati
on, CN=Microsoft Corporation Third Party Marketplace Root
Validity
Not Before: Jun 27 21:22:45 2011 GMT
Not After : Jun 27 21:32:45 2026 GMT
Subject: C=US, ST=Washington, L=Redmond, O=Microsoft Corpor

ation, CN=Microsoft Corporation UEFI CA 2011

Subject Public Key Info:

Public Key Algorithm: rsaEncryption

Public-Key: (2048 bit)

Modulus:

00:a5:08:6c:4c:c7:45:09:6a:4b:0c:a4:c0:87:7f:

06:75:0c:43:01:54:64:e0:16:7f:07:ed:92:7d:0b:

b2:73:bf:0c:0a:c6:4a:45:61:a0:c5:16:2d:96:d3:

===== certs_dir/certs_dir-1.der =====

subject=C=US, O=Microsoft Corporation, CN=Microsoft UEFI CA 2023

issuer=C=US, O=Microsoft Corporation, CN=Microsoft RSA Devices Root CA 2021

Certificate:

Data:

Version: 3 (0x2)

Serial Number:

33:00:00:00:16:36:bf:36:89:9f:15:75:cc:00:00:00:00:16

Signature Algorithm: sha256WithRSAEncryption

Issuer: C=US, O=Microsoft Corporation, CN=Microsoft RSA Devices Root CA 2021

Validity

Not Before: Jun 13 19:21:47 2023 GMT

Not After : Jun 13 19:31:47 2038 GMT

Subject: C=US, O=Microsoft Corporation, CN=Microsoft UEFI CA 2023

Subject Public Key Info:

Public Key Algorithm: rsaEncryption

Public-Key: (2048 bit)

Modulus:

00:bd:22:2a:ae:ef:1a:31:85:13:78:51:a7:9b:fd:

fc:78:d1:63:b8:1a:9b:63:f5:12:06:db:4b:41:35:

6a:6f:ab:f5:6a:04:cc:97:cf:bb:d4:08:09:1a:61:

===== certs_dir/certs_dir-2.der =====

subject=C=US, ST=Washington, L=Redmond, O=Microsoft Corporation, CN=Microsoft Windows Production PCA 2011

issuer=C=US, ST=Washington, L=Redmond, O=Microsoft Corporation,

CN=Microsoft Root Certificate Authority 2010
Certificate:
Data:
Version: 3 (0x2)
Serial Number:
61:07:76:56:00:00:00:00:08
Signature Algorithm: sha256WithRSAEncryption
Issuer: C=US, ST=Washington, L=Redmond, O=Microsoft Corporation, CN=Microsoft Root Certificate Authority 2010
Validity
Not Before: Oct 19 18:41:42 2011 GMT
Not After : Oct 19 18:51:42 2026 GMT
Subject: C=US, ST=Washington, L=Redmond, O=Microsoft Corporation, CN=Microsoft Windows Production PCA 2011
Subject Public Key Info:
Public Key Algorithm: rsaEncryption
Public-Key: (2048 bit)
Modulus:
00:dd:0c:bb:a2:e4:2e:09:e3:e7:c5:f7:96:69:bc:
00:21:bd:69:33:33:ef:ad:04:cb:54:80:ee:06:83:
bb:c5:20:84:d9:f7:d2:8b:f3:38:b0:ab:a4:ad:2d:
===== certs_dir/certs_dir-3.der =====
subject=C=US, O=Microsoft Corporation, CN=Windows UEFI CA 2023
issuer=C=US, ST=Washington, L=Redmond, O=Microsoft Corporation,
CN=Microsoft Root Certificate Authority 2010
Certificate:
Data:
Version: 3 (0x2)
Serial Number:
33:00:00:00:1a:88:8b:98:00:56:22:84:c1:00:00:00:00:00:1a
Signature Algorithm: sha256WithRSAEncryption
Issuer: C=US, ST=Washington, L=Redmond, O=Microsoft Corporation, CN=Microsoft Root Certificate Authority 2010
Validity
Not Before: Jun 13 18:58:29 2023 GMT
Not After : Jun 13 19:08:29 2035 GMT

Subject: C=US, O=Microsoft Corporation, CN=Windows UEFI CA 2023

Subject Public Key Info:

Public Key Algorithm: rsaEncryption

Public-Key: (2048 bit)

Modulus:

00:bc:b2:35:d1:54:79:b4:8f:cc:81:2a:6e:b3:12:

d6:93:97:30:7c:38:5c:bf:79:92:19:0a:0f:2d:0a:

fe:bf:e0:a8:d8:32:3f:d2:ab:6f:6f:81:c1:4d:17:

▼ step 2

This Microsoft CA cert is not the root — it's signed by Microsoft's UEFI CA.

- **Platform Key (PK):** links the platform firmware with its owner. The key owner can create new Key Exchange Keys (KEKs) and transfer firmware ownership.

Other vars like KEK/db/dbx: define allowed and forbidden binaries.

- **KEK (Key Exchange Key):** This establishes a trust relationship between the operating system and the platform firmware.
- **db (Database):** The `db` is a collection of trusted signatures. EFI executable files are allowed to run only if their signatures match those listed in the `db`.
- **dbx (Database of Forbidden Signatures):** The `dbx` contains signatures of executables that should not run, even if they match an entry in the `db`.
- **MoKLists:** This stores the list of Machine Owner Keys (MoKs). At the shim level, MoKs are used to validate drivers and the kernel, while the `db` keys perform validation during the initial boot stage.

Task 2 - SHIM

▼ step 3

Kali Linux does **not use shim by default**. The installer sets GRUB (`grubx64.efi`) directly as the bootloader.

BRUB is not signed by Microsoft, but it does not stops Kali to boot via UEFI (skips Microsoft part).

`/boot/efi/EFI/kali/grubx64.efi` - path

```
(nikita@nikita)-[~]
$ efibootmgr -v
BootCurrent: 0003
Timeout: 0 seconds
BootOrder: 0003,0000,0001,0002,0004
Boot0000* UiApp FvVol(7cb8bdc9-f8eb-4f34-aaea-3ee4af6516a1)/FvFile(462caa21-76
14-4503-836e-8ab6f4662331)
    dp: 04 07 14 00 c9 bd b8 7c eb f8 34 4f aa ea 3e e4 af 65 16 a1 / 04 06
14 00 21 aa 2c 46 14 76 03 45 83 6e 8a b6 f4 66 23 31 / 7f ff 04 00
Boot0001* UEFI VBOX HARDDISK VBF92ef0f0-a7397a39 PciRoot(0x0)/Pci(0xd,0
x0)/Sata(0,65535,0){auto_created_boot_option}
    dp: 02 01 0c 00 d0 41 03 0a 00 00 00 00 / 01 01 06 00 00 0d / 03 12 0a 0
0 00 00 ff ff 00 00 / 7f ff 04 00
    data: 4e ac 08 81 11 9f 59 4d 85 0e e2 1a 52 2c 59 b2
Boot0002* UEFI PXEv4 (MAC:0800278A2480) PciRoot(0x0)/Pci(0x3,0x0)/MAC(0800278a
2480,1)/IPV4(0.0.0.0,0,DHCP,0.0.0.0,0.0.0.0,0.0.0.0){auto_created_boot_option}
    dp: 02 01 0c 00 d0 41 03 0a 00 00 00 00 / 01 01 06 00 00 03 / 03 0b 25 0
0 08 00 27 8a 24 80 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 01 / 03 0c 1b 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 / 7f ff 04 00
    data: 4e ac 08 81 11 9f 59 4d 85 0e e2 1a 52 2c 59 b2
Boot0003* Kali HD(1,GPT,35e95836-4d05-441c-89c5-fbcf63173c3b,0x800,0x1e3000)/
\EFI\kali\grubx64.efi
    dp: 04 01 2a 00 01 00 00 00 00 08 00 00 00 00 00 00 30 1e 00 00 00 00
00 36 58 e9 35 05 4d 1c 44 89 c5 fb cf 63 17 3c 3b 02 02 / 04 04 30 00 5c 00
45 00 46 00 49 00 5c 00 6b 00 61 00 6c 00 69 00 5c 00 67 00 72 00 75 00 62 00
78 00 36 00 34 00 2e 00 65 00 66 00 69 00 00 00 / 7f ff 04 00
Boot0004* UEFI PXEv6 (MAC:0800278A2480) PciRoot(0x0)/Pci(0x3,0x0)/MAC(0800278a
2480,1)/IPV6([::],0,Static,[::],[::],64){auto_created_boot_option}
    dp: 02 01 0c 00 d0 41 03 0a 00 00 00 00 / 01 01 06 00 00 03 / 03 0b 25 0
0 08 00 27 8a 24 80 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 01 / 03 0d 3c 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 40
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 / 7f ff 04 00
    data: 4e ac 08 81 11 9f 59 4d 85 0e e2 1a 52 2c 59 b2
(nikita@nikita)-[~]
```

▼ step 4

I have checked on shim on kali, but only `grubx64` and only kali folder - so no shimx and no ubuntu folder.

```
(nikita@nikita)-[/boot]
$ sudo ls -l /boot/efi/EFI
total 4
drwx----- 2 root root 4096 Sep 15 09:34 kali

(nikita@nikita)-[/boot]
$ sudo ls -l /boot/efi/EFI/kali
total 144
-rwx----- 1 root root 147456 Sep 15 09:34 grubx64.efi
```


grubx64 is not signed

```
ts: cannot access /efi : No such file or directory
(nikita@nikita)-[/boot]
$ sudo sbverify --list /boot/efi/EFI/kali/grubx64.efi
No signature table present

(nikita@nikita)-[/boot]
$ sudo design -S -i /boot/efi/EFI/kali/grubx64.efi
No signatures found.
```

The GRUB bootloader is not signed by any certification authority (CA), as this Kali VM does not use Secure Boot or shim.

▼ step 5

stored in the `Attribute Certificate Table` section of the binary [\[doc\]](#)

▼ step 6

It's defined in the PKCS #7 format

P.S. The signature data is stored in the **X.509** format. X.509 is the standard for digital certificates, that contains the public key infrastructure (PKI) signature of the binary, ensuring Authenticity (CIA).

▼ step 7

Installing `pyew` tool and dependencies.

```

git version 2.17.1
(nikita@nikita)-[~]
$ git clone https://github.com/joxeankoret/pyew.git
Cloning into 'pyew' ...
remote: Enumerating objects: 806, done.
remote: Total 806 (delta 0), reused 0 (delta 0), pack-reused 806 (from 1)
Receiving objects: 100% (806/806), 2.63 MiB | 106.00 KiB/s, done.
Resolving deltas: 100% (376/376), done.
(nikita@nikita)-[~]
$ cd pyew
(nikita@nikita)-[~/pyew]
$ sudo apt-get install python3 python3-pip python3-dev libpcap-dev
Reading package lists ... Done
Building dependency tree ... Done
Reading state information ... Done
The following additional packages will be installed:
  libcap-dev libcap2 libcap2-bin libdbus-1-dev libnss-systemd libpam-systemd
  libpkgconf3 libpython3-dev libpython3-stdlib libpython3.13 libpython3.13

```

```

(nikita@nikita)-[~/pyew]
$ sudo pip2 install setuptools wheel
DEPRECATION: Python 2.7 reached the end of its life on January 1st, 2020. Please upgrade
r Python as Python 2.7 is no longer maintained. pip 21.0 will drop support for Python
January 2021. More details about Python 2 support in pip can be found at https://pip
o/en/latest/development/release-process/#python-2-support pip 21.0 will remove support
his functionality.
Collecting setuptools
  Downloading setuptools-44.1.1-py2.py3-none-any.whl (583 kB)
    | 583 kB 274 kB/s
Requirement already satisfied: wheel in /usr/local/lib/python2.7/dist-packages (0.37
Installing collected packages: setuptools
Successfully installed setuptools-44.1.1
(nikita@nikita)-[~/pyew]
$ sudo pip2 install capstone
DEPRECATION: Python 2.7 reached the end of its life on January 1st, 2020. Please upgrade
r Python as Python 2.7 is no longer maintained. pip 21.0 will drop support for Python
January 2021. More details about Python 2 support in pip can be found at https://pip
o/en/latest/development/release-process/#python-2-support pip 21.0 will remove support
his functionality.
Collecting capstone
  Using cached capstone-5.0.1.tar.gz (2.9 MB)
Building wheels for collected packages: capstone
  Building wheel for capstone (setup.py) ... done
  Created wheel for capstone: filename=capstone-5.0.1-py2-none-manylinux1_x86_64.whl
72721 sha256=5244206f1306aef206475588587676847dbc3c1c9d8a2019bce4dff02eceece
  Stored in directory: /root/.cache/pip/wheels/1f/55/36/dae037be731eca3b5b9d76a4e8a0
0bc5d538675c5a
Successfully built capstone
Installing collected packages: capstone
Successfully installed capstone-5.0.1

```

```

File Actions Edit View Help
└─$ sudo python2 pyew.py /boot/efi/EFI/kali/grubx64.efi
PE Information

64 Bits binary
Sections:
.text 0x1000 0xd000 53248
.data 0xe000 0x10000 65536
.mods 0x1e000 0x5000 20480
.reloc 0x23000 0x1000 4096

Entry Point at 0x1000
Virtual Address is 0x1000
***Error loading imports PE instance has no attribute 'DIRECTORY_ENTRY_IMPORT'
Code Analysis ...
Analyzing address 0x00006574 - 7 in queue / 177 total
0000 4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00  MZ.....
0010 B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00  .....@.....
0020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
0030 00 00 00 00 00 00 00 00 00 00 00 00 80 00 00 00  .....
0040 0E 1F BA 0E 00 84 09 CD 21 B8 01 4C CD 21 54 68  .....!..L!Th
0050 69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F  is program canno
0060 74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20  t be run in DOS
0070 6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00  mode....$.
0080 50 45 00 00 64 86 04 00 00 8E A4 54 00 00 00 00  PE..d.....T....
0090 00 00 00 00 F0 00 0E 02 0B 02 00 00 00 D0 00 00  .....
00A0 00 50 01 00 00 00 00 00 10 00 00 00 10 00 00  .P.....
00B0 00 00 00 00 00 00 00 00 10 00 00 00 10 00 00  .....
00C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
00D0 00 40 02 00 00 10 00 00 00 00 00 00 0A 00 00 00  .@.....
00E0 00 00 01 00 00 00 00 00 00 00 01 00 00 00 00 00  .....
00F0 00 00 01 00 00 00 00 00 00 00 01 00 00 00 00 00  .....
0100 00 00 00 00 10 00 00 00 00 00 00 00 00 00 00 00  .....
0110 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
0120 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
0130 00 30 02 00 00 10 00 00 00 00 00 00 00 00 00 00  .0.....
0140 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
0150 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
0160 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
0170 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
0180 00 00 00 00 00 00 00 00 2E 74 65 78 74 00 00 00  .....text...
0190 00 D0 00 00 00 10 00 00 00 D0 00 00 00 10 00 00  .....
01A0 00 00 00 00 00 00 00 00 00 00 00 00 20 00 00 60  .....
01B0 2E 64 61 74 61 00 00 00 00 01 00 00 E0 00 00 00  .data.....
01C0 00 00 01 00 00 E0 00 00 00 00 00 00 00 00 00 00  .....
01D0 00 00 00 00 40 00 00 C0 6D 6F 64 73 00 00 00 00  ....@... mods....
01E0 00 50 00 00 00 E0 01 00 00 50 00 00 00 E0 01 00  .P.....P.....

```

Type `pyew.pe.OPTIONAL_HEADER.DATA_DIRECTORY` to get a listing of all data directory entries.

```

[0x00000000]> pyew.pe.OPTIONAL_HEADER.DATA_DIRECTORY
[<Structure: [IMAGE_DIRECTORY_ENTRY_EXPORT] 0x108 0x0 VirtualAddress: 0x0 0x10C 0x4 Size: 0x0>,
<Structure: [IMAGE_DIRECTORY_ENTRY_IMPORT] 0x110 0x0 VirtualAddress: 0x0 0x114 0x4 Size: 0x0>,
<Structure: [IMAGE_DIRECTORY_ENTRY_RESOURCE] 0x118 0x0 VirtualAddress: 0x0 0x11C 0x4 Size: 0x0>,
<Structure: [IMAGE_DIRECTORY_ENTRY_EXCEPTION] 0x120 0x0 VirtualAddress: 0x0 0x124 0x4 Size: 0x0>,
<Structure: [IMAGE_DIRECTORY_ENTRY_SECURITY] 0x128 0x0 VirtualAddress: 0x0 0x12C 0x4 Size: 0x0>,
<Structure: [IMAGE_DIRECTORY_ENTRY_BASERELOC] 0x130 0x0 VirtualAddress: 0x23000 0x134 0x4 Size: 0x1000>,
<Structure: [IMAGE_DIRECTORY_ENTRY_DEBUG] 0x138 0x0 VirtualAddress: 0x0 0x13C 0x4 Size: 0x0>,
<Structure: [IMAGE_DIRECTORY_ENTRY_COPYRIGHT] 0x140 0x0 VirtualAddress: 0x0 0x144 0x4 Size: 0x0>,
<Structure: [IMAGE_DIRECTORY_ENTRY_GLOBALPTR] 0x148 0x0 VirtualAddress: 0x0 0x14C 0x4 Size: 0x0>,
<Structure: [IMAGE_DIRECTORY_ENTRY_TLS] 0x150 0x0 VirtualAddress: 0x0 0x154 0x4 Size: 0x0>,
<Structure: [IMAGE_DIRECTORY_ENTRY_LOAD_CONFIG] 0x158 0x0 VirtualAddress: 0x0 0x15C 0x4 Size: 0x0>,
<Structure: [IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT] 0x160 0x0 VirtualAddress: 0x0 0x164 0x4 Size: 0x0>,
<Structure: [IMAGE_DIRECTORY_ENTRY_IAT] 0x168 0x0 VirtualAddress: 0x0 0x16C 0x4 Size: 0x0>,
<Structure: [IMAGE_DIRECTORY_ENTRY_DELAY_IMPORT] 0x170 0x0 VirtualAddress: 0x0 0x174 0x4 Size: 0x0>,
<Structure: [IMAGE_DIRECTORY_ENTRY_COM_DESCRIPTOR] 0x178 0x0 VirtualAddress: 0x0 0x17C 0x4 Size: 0x0>,
<Structure: [IMAGE_DIRECTORY_ENTRY_RESERVED] 0x180 0x0 VirtualAddress: 0x0 0x184 0x4 Size: 0x0>]
[0x00000000]>

```

<
Structure: [IMAGE_DIRECTORY_ENTRY_SECURITY] 0x128 0x0

```
VirtualAddress: 0x0 0x12C 0x4  
Size: 0x0  
>
```

The conclusion:

- The **virtual address** where the signature data should is **invalid** (starts with `0x0`)
- There is **no signature** present in the binary (size `0x0`).

▼ step 8

Since there's **no signature** in the GRUB binary, Task 8 cannot be completed for **grubx64.efi**.

1. Microsoft Corporation Third Party Marketplace Root
→ (signs)
2. Microsoft Corporation UEFI CA 2011
→ (signs)
3. Microsoft Windows UEFI Driver Publisher

Task 3 - GRUB (BONUS)

▼ step 9

Inspect grub binary for signatures:

```
sudo sbverify --list /boot/efi/EFI/ubuntu/grubx64.efi
```

```
Can't open image /boot/efi/EFI/ubuntu/grubx64.efi  
  
(nikita@nikita)-[~/pyew]  
$ sudo sbverify --list /boot/efi/EFI/kali/grubx64.efi  
No signature table present  
  
(nikita@nikita)-[~/pyew]  
$
```

▼ step 10

The X.509 certificate used to verify the GRUB signature is stored in shim because shim is the first bootloader that is verified by the UEFI firmware.

Once shim is verified (via its signature), it can trust the X certificate that it contains to verify further stages like GRUB.

Reasons for Security CIA:

- If an attacker attempts to alter the shim binary or the embedded certificate, it will be detected during the boot process because the firmware verifies shim's signature before allowing the system to proceed.
- The certificate embedded in shim is protected from external modifications because shim is signed by a trusted authority (e.g., Microsoft UEFI CA), and shim's signature ensures the integrity of the embedded certificate.

▼ step 11

Canonical Ltd. Secure Boot Signing. cause usually they sign grub bootloader and also certificate should be embedded in shim.

▼ step 12

```
(nikita@nikita)-[~/pyew]
$ sudo binwalk -e --run-as=root /boot/efi/EFI/kali/grubx64.efi
DECIMAL      HEXADECIMAL  DESCRIPTION
WARNING: One or more files failed to extract: either no utility was found or it's unimple
```

So, again no shim, no results. But ideally it would be next steps:

`sudo dd` - for extracting certificates

then after obtaining them I would use `openssl x509` to observe it

▼ step 13

Here I should have been confirmed via **sbverify** that **Canonical's certificate** signed **GRUB**.

